

第 11 回事前課題

作業ログ (第 10 週末)

報告者	櫛濱 和輝 (25G1046)
報告日	2026 年 7 月 7 日
チーム	チーム 9
プロジェクト名	赤外線通信で歌うカエル
担当パート	今週は本来の親機/ピアノ担当から少し外れた作業が中心. 手代木が諦めた LCD (顔+歌詞の 2 枚表示) の引き取り検討と実装, アンケート用のピアノ音の収録, IR 通信の伝搬遅延を測るための計測プログラム (hack-koP2.ino の kokitest 版) の整備
本来の今週の位置づけ	第 10 週末 (7/7) = 最終週. 計画書 v1.1 (2026/6/19 版) のガント上, 第 10 週は 510: 結合テストの最終期間. 前回個人作業ログ (7/1) で次回計画に挙げた MOP1 演奏タイミング誤差と MOP2 赤外線通信成功率の定量計測に向けて計測プログラムを準備しつつ, MOP5 に関わる歌詞/顔の LCD 表示を手代木から引き取った
報告対象	前回個人作業ログ (7/1) 以降に手を付けた 3 件. 手代木担当だった LCD 表示子機 (SUNLIKE SC1602B を 2 枚使って顔と歌詞を出す) の実装, アンケート用のピアノ音の収録, IR 伝搬遅延の計測プログラム (親機 GPIO パルスを基準に <code>micros()</code> を 2 点で取って差分を測る仕組み) の整備. LCD は実装まで持っていたが実機で安定して動かず, 原因の切り分けが済んでいない
作業時間	計 8 時間

1. 進捗サマリー

現在地は第 10 週末 (7/7) で, ガント上は 510: 結合テストの最終期間. 前回 (7/1) までに親機の BPM ポテンショメータ化と 4 子機の輪唱までは動いていて, 今週は残っていた歌詞/顔の表示 (LCD) と, MOP1/MOP2 を数字で示すための計測の準備に回った. 正直に書くと, 今週の 3 件のうち LCD はまだ動いていない.

- LCD (顔+歌詞) の引き取りと実装: もともと LCD は手代木の担当だったが, 本人が「うまく映らないので諦める」と言って手を離したので, 俺が引き取って検討した. 顔を出す LCD1 と歌詞を出す LCD2 の 2 枚を 1 台の Arduino にぶら下げて, RS とデータ線 (DB4-7) は 2 枚で共有し, E 線だけ個別 (LCD1→D11, LCD2→D10) にして書き分ける構成にした. スケッチ (`flog_kao_kasi_ir_full`) は一通り書き上げて, IR で受けた tick に合わせて歌詞を 1 文字ずつ進めつつ顔を口パクさせる, というところまで実装した. **ただ実機ではまだ安定して映ら**

ない。表示が出ない/薄い/片方だけ映る、といった症状が出ていて、配線なのかコントラスト (V0) なのか、あるいは書き込みタイミングの問題なのか、原因をまだ切り分けられていない。今週いちばん手こずっているのがここ

- ピアノ音の収録：MOP4（音色の再現性）のアンケートに使うため、Processing (hackko.pde) で鳴らしている改善後のピアノ音を収録した。聴衆に聞かせて「ピアノだと分かるか」を問う設問で使う想定。収録自体は問題なく済んだ
- IR 伝搬遅延の計測プログラム：MOP1（演奏タイミング誤差）と MOP2（通信成功率）を数字で出すために、IR の伝搬遅延を測る仕組みを子機側 (hack-koP2.ino の kokitest 版) に組んだ。親機の GPIO をワイヤーで子機の割り込みピンに直結し、その立ち上がりを計測開始トリガにする。親機はトリガを出した直後に `sendNEC()` で IR を送り、子機が IR をデコードし終えた時刻との差分を取ることで「NEC を受けてデコードして loop で処理するまで」に何 μs かかっているかが分かる。この差分が通信誤差の主成分になる、という筋書き
- 残課題：LCD が動かない件の原因切り分けが最優先。計測プログラムは組めたので、あとは実際に流してデータを取れば MOP1/MOP2 の数字が出せる。前回から持ち越している双子音処理の `delay()` 非ブロック化には今週は手が回らなかった

2. ガントチャート

第 10 週末時点でも計画書 v1.1 (2026/6/19 版) から計画自体の変更はないので、チーム全体/個人とも v1.1 のガントチャートを掲載する。着色セル (■) は当該タスクの実施週を表す。今週は第 10 週 (最終週) にあたり、510: 結合テストの締め期間になる。

2.1 チーム全体ガントチャート

第 3 層：活動タスク	担当者	5 週	6 週	7 週	8 週	9 週	10 週
110: ピアノ音の作成	櫛濱・久家	■		■			
120:arduino との通信用関数 (ピアノ)	櫛濱・久家	■					
130: フルート音の作成	家田	■					
140:arduino との通信用関数 (フルート)	家田	■					
150: トランペット音の作成	木下	■					
160:arduino との通信用関数 (トランペット)	木下	■					
170: ドラム音の作成 (キック)	久家		■				
171: ドラム音の作成 (シンバル)	久家		■				
180:arduino との通信用関数 (ドラム)	久家		■				

210: ピアノのプログラム	櫛濱						
220: フルートのプログラム	家田						
230: トランペットのプログラム	木下						
240: ドラムのプログラム	久家						
250: 楽譜進行プログラム	渡邊						
310: 赤外線管理プログラム	家田・渡邊						
320: テンポ管理プログラム	渡邊						
410: 回路作成	手代木						
420: LED マトリクス作成	手代木						
510: 結合テスト	全員						

2.2 個人ガントチャート（計画書 v1.1 から櫛濱担当分を抜粋）

2.1 節のチーム全体ガントから櫛濱担当分を抜き出した個人計画。第 10 週は 510: 結合テストの最終期間にあたる。計画上、歌詞/顔の LCD 表示（420 系）は本来手代木の担当だが、今週は手が止まっていたぶんを引き取って動かそうとした。結果としてはまだ表示が安定せず、最終週に持ち込んでしまっている。

管理	第 2 層	第 3 層タスク	5 週	6 週	7 週	8 週	9 週	10 週
110	100 (Processing)	ピアノ音の作成						
120	100 (Processing)	arduino との通信用関数（ピアノ）						
170	100 (Processing)	ドラム音の作成（キック）						
210	200 (子機)	ピアノのプログラム						
510	500 (包括)	結合テスト（全員）						

2.3 今週の作業位置づけ

- 現在地は第 10 週末（7/7）で、最終週。GC 上は 510: 結合テストの締め期間にあたる。前回（7/1）までに親機のテンポ連続可変化と 4 子機の輪唱までは動いているので、今週は残りの表示系（LCD）と、これまで聴感頼みだった MOP1/MOP2 を数字にするための計測に注力した
- 手代木の担当だった LCD 表示（420 系，MOP5 の歌詞/アニメ同期に対応）を引き取り、顔＋歌詞の 2 枚表示子機として実装した。ただし実機で安定表示できておらず、最終週に持ち込んで

いる

- MOP1/MOP2 の定量評価に向けて、IR 伝搬遅延の計測プログラムを子機側に組んだ。親機 GPIO パルスを計測開始トリガに使い、IR デコード完了までの時間を `micros()` 差分で取る
- 最終週の残り時間は、LCD が映らない原因の切り分け、計測プログラムでの実測 (MOP1/MOP2)、アンケート (MOP4) の実施、そして最終発表のデモ準備に充てる

3. 作業ログ

作業項目	開始日	予定完了日	状態	進捗率	依存関係・ブロッカー	コメント
LCD 表示子機の配線設計 (顔+歌詞 2 枚)	7/1	7/2	完了	100%	SC1602B×2 / CHQ1838D	手代木から引き取り、RS(D12) とデータ線 (D5,D4,D3,D2) を 2 枚で共有し、E 線だけ LCD1→D11・LCD2→D10 に分けて書き分ける配線に決めた。SC1602B はピン 1=VDD/ピン 2=VSS で市販の 1602A と電源が逆なので、そこだけ注意して配線表 (HAISEN.md) にまとめた
LCD 表示子機スケッチの実装 (flog_kao_kasi_ir_full)	7/2	7/4	進行中	70%	LiquidCrystal / IRremote 4.x	顔 (LCD1) と歌詞 (LCD2) を LiquidCrystal lcd1(12,11,5,4,3,2) / lcd2(12,10,5,4,3,2) で分けて、IR で受けた tick に合わせて歌詞を 1 文字ずつ進め、顔を口パクさせる実装まで書いた。コードは通るが実機で安定して映らず、原因切り分けが未了。最終週に持ち越し
ピアノ音の収録 (アンケート用)	7/3	7/3	完了	100%	hackko.pde (改善後音色)	MOP4 の音色認識アンケートに使うため、Processing で鳴らしている改善後のピアノ音を収録した。「これはピアノか」を聴衆に問う設問の音源として使う
IR 伝搬遅延の計測プログラム整備 (hack-koP2 kokitest 版)	7/4	7/6	完了	100%	attachInterrupt / 親機 GPIO 直結線	親機 GPIO を子機の PIN_SYNC_IN に直結し、立ち上がり割り込み onSyncRise() で syncRiseUs=micros() を記録 (計測開始)。IR デコード後に recvUs=micros() を取り、差分で「NEC 受信+デコード+loop 処理」の所要時間を測る
tickLow 絶対同期の受信確認	7/6	7/6	完了	100%	hack_oya_kai2	親機 hack_oya_kai2_2 が IR の address に tickCount 下位 8bit を乗せて送る方式に合わせ、計測版子機でも取りこぼし後に位置が自己修復することを確認した

4. 評価事項

計画書 v1.1 (2026/6/19 版) の V&V 計画のうち、今週の作業 (LCD 表示の引き取り/ピアノ音収録/IR 伝搬遅延の計測プログラム) に対応する項目を整理する。今週は数字を取る手前の「計測の準備」と、表示系の「実装したが動かない」状態が中心なので、達成というより地ならしの週になった。

4.1 対応する有効指標 MOE

No.	MOE	評価基準
MOE1	違和感なく聞くことができる	輪唱が聴衆にズレを感じさせることなく演奏できている
MOE3	楽器音らしい音色である	Processing で合成した音が楽器音として聴衆に認識される (アンケートで確認)
MOE4	歌詞/顔の表示で演出が伝わる	LCD の歌詞と顔の表示が演奏に同期し、カエルが歌っている演出として伝わる

4.2 対応する性能指標 MOP (計画書 v1.1)

No.	性能指標	目標値・評価基準
MOP1	演奏タイミング誤差	各演奏側 Arduino 間の音符開始タイミングのズレが 30 ms 以内であること
MOP2	同期信号の受信成功率	指揮側から送信した赤外線信号を演奏側が正しく受信できる割合が 95% 以上であること
MOP4	音色の再現性	倍音構造/ADSR の設定により、対応する楽器の音色として聴衆の過半数に評価されること
MOP5	歌詞/アニメーション同期遅延	歌詞と顔の表示が演奏の tick に対して遅れなく追従すること

4.3 今週の自己評価

No.	項目	達成状況	備考
MOP1	演奏タイミング誤差	計測準備 OK	伝搬遅延の計測プログラムを整備し, <code>micros()</code> 差分で遅延を取れる状態にした。実測データの取得はこれから

MOP2	同期信号受信成功率	計測準備OK	同じ計測版子機で受信/デコードのログが取れるので、一定時間流せば成功率を算出できる．実測はこれから
MOP4	音色の再現性	準備中	アンケート用にピアノ音を収録済み．聴衆過半数の評価はアンケート実施待ち
MOP5	歌詞/アニメ同期	未達(難航)	顔+歌詞の2枚LCDを実装したが実機で安定表示できず．原因切り分けが済んでおらず，最終週に持ち越し

4.4 今後評価予定の項目（参考）

- MOP1：計測版子機を使って親機 GPIO パルスから IR デコード完了までの遅延を `micros()` で収集し，各子機間の音符開始タイミングのズレが 30ms 以内かを数字で確認する（実測これから）
- MOP2：計測版子機を一定時間動かし，受信/デコードの成功・失敗ログから成功率を算出して 95% 目標と比較する（実測これから）
- MOP4：収録したピアノ音を含む各楽器の音色認識アンケートを実施し，聴衆過半数が識別できるかを確認する
- MOP5：LCD が安定して映るようになったら，歌詞/顔の表示が演奏 tick に遅れなく追従するかを聴感と目視で確認する（まず表示が出ることが前提）

5. 課題管理

課題	発生原因	影響度	優先度	対策	期限	状態
LCD が実機で安定して映らない	表示が出ない/薄い/片方だけ映る等．配線・コントラスト (V0)・書き込みタイミングのどれが主因か切り分けできていない	高	高	V0(コントラスト)→RS(D12)→共有データ線(D5,D4,D3,D2)→E線(D11/D10)の順で1つずつ潰す．V0をGND直結にして最濃で切り分ける手も試す	7/8	未対応
演奏タイミング誤差が未計測 (MOP1)	計測プログラムは整備したが実データをまだ取っていない	高	高	計測版子機を4台で動かし <code>micros()</code> 差分を収集して 30ms 以内かを確認	7/8	未対応
赤外線通信成功率が未計測 (MOP2)	95% 目標との定量比較が未実施	高	高	計測版子機の実受信/デコードログから成功率を算出	7/8	未対応
音色認識アンケート未実施 (MOP4)	聴衆過半数の評価がまだ得られていない	中	中	収録したピアノ音を含め各楽器の音色認識テストを実施	7/8	未対応
双子音処理の <code>delay()</code> による IR 取りこぼし	DOUBLE_START 以降の半 tick 遅延を <code>delay()</code> で実装しており，その間 IR 受信が止まる	中	中	<code>millis()</code> ベースの非ブロック化（前回から継続，今週は未着手）	7/8	未対応

※影響度＝プロジェクトへのインパクト

※優先度＝対応の緊急性

6. AI 利用状況と検証

6.1 利用内容

- 利用目的：LCD（顔＋歌詞の 2 枚表示）子機スケッチの実装，IR 伝搬遅延の計測プログラムの整備
- 入力（プロンプト要旨）：
 - SC1602B を 2 枚使って，1 枚に顔・もう 1 枚に歌詞を出したい．RS とデータ線は共有して E 線だけ分ける配線でいけるか
 - IR で受けた tick に合わせて歌詞を 1 文字ずつ進め，顔を口パクさせたい
 - IR 通信の伝搬遅延を測りたい．親機の GPIO を子機に直結して立ち上がりを割り込みで拾い，そこから IR をデコードし終わるまでの時間を `micros()` で測る形にしたい
- 出力概要：
 - `flog_kao_kasi_ir_full.ino`：LiquidCrystal lcd1(12,11,5,4,3,2) / lcd2(12,10,5,4,3,2) で 2 枚を E 線切替で書き分け，IR 受信 tick に合わせて歌詞送り・口パク・顔の移動を行う実装
 - `hack-koP2.ino`（kokitest 版）：PIN_SYNC_IN の立ち上がり割り込み `onSyncRise()` で `syncRiseUs=micros()` を記録し，IR デコード後の `recvUs=micros()` との差分を出力する計測ロジック

6.2 検証方法

- 検証手法：
 - LCD：書き込み直後に LCD1 の待機顔と LCD2 の待機メッセージ（MATTETA KAERU / OSHITE START!）が出るかを目視で確認．出ない/薄いときは V0 を回してコントラストを振る．**現状ここが安定せず，表示が出たり出なかったりで再現性が取れていない**
 - LCD：片方だけ映る症状のときは，映らない側の E 線（LCD1=D11 / LCD2=D10）の接触を最優先で疑う（配線表の切り分け手順に沿って確認中）
 - 計測プログラム：親機 GPIO を子機の PIN_SYNC_IN に直結し，親機がトリガを出した直後に `sendNEC()` を呼ぶ．Serial Monitor（115200 baud）で `syncRiseUs` と `recvUs` の差分が μs 単位で出力されることを確認
 - 計測プログラム：GPIO は電気信号なので子機の割り込みはほぼ遅延 0 で入る．したがって差分は IR 経路（NEC フレーム送出＋受信＋デコード＋loop 検出）の実効遅延を表す，という前提が成り立つことを確認
- 参照資料：
 - チーム計画書/設計書 v1.1（2026/6/19 版）
 - 前回個人作業ログ（第 10 回事前課題，2026/7/1）

- flog_kao_kasi_ir_full の配線・動作確認ガイド (HAISEN.md)
- Arduino 公式ドキュメント (`attachInterrupt()` / `micros()` / `LiquidCrystal` ライブラリ)
- IRremote 4.x ライブラリ (`IRremote.hpp`) のドキュメント
- 検証手順：
 1. 表示子機 (`flog_kao_kasi_ir_full`) を音楽子機とは別の Arduino に書き込み、LCD1/LCD2 に待機表示が出るかを見る
 2. 表示が出ないときは V0 を GND 直結にして最濃で映るか、配線が原因かコントラストが原因かを切り分ける (この切り分けが今週まだ終わっていない)
 3. 計測版子機 (`hack-koP2.ino` kokitest 版) を書き込み、親機 GPIO を `PIN_SYNC_IN` に直結する
 4. 親機からトリガ + `sendNEC()` を出し、Serial Monitor で `recvUs-syncRiseUs` が妥当な μs 値で出るかを確認する
 5. 4 子機ぶんの遅延を並べて、子機間のばらつき (MOP1) と受信成功率 (MOP2) を集計する (本実測は最終週に実施)

6.3 ファクトチェック結果

- 一致点：
 - 計測の起点に GPIO 直結の立ち上がり割り込みを使う設計は妥当で、電気信号の伝搬は μs オーダーで無視できるため、差分を IR 経路の遅延とみなせることを確認した
 - `LiquidCrystal` で RS とデータ線を共有し E 線だけ分ける 2 枚構成は、HD44780 互換 LCD の一般的な使い方として成立することを確認した (E をトグルするタイミングで書き込み先が決まる)
- 不一致/疑義：
 - LCD が安定して映らない原因が特定できていない。コード上のロジックは通っているので、主因はソフトよりハード側 (コントラスト、E 線や共有データ線の接触、SC1602B の電源ピンの向き) の可能性が高いと見ているが、まだ確証がない
 - 計測差分には NEC フレーム自体の送出時間 (プロトコル上の固定時間) も含まれる。純粹な「受信 + デコード + loop」だけを切り出すには、フレーム送出時間を別途差し引く補正が要るかもしれない (最終週に実データを見て判断)
- 最終判断：計測プログラムは採用 (MOP1/MOP2 の定量化の土台として妥当)。LCD は保留 (実機で映る状態にしてから再評価)
- 根拠：
 - GPIO トリガ基準の `micros()` 差分計測は、これまで聴感頼みだった演奏タイミング誤差と通信成功率を、初めて数字で扱える形にできる
 - LCD はコードとしては書けているが、実機で表示が安定しない以上「動く」とは言えないため、原因切り分けが済むまで達成扱いにはしない

7. 次回計画

- 目標（最終週の残り／最終発表まで）：
 - － LCD が実機で安定して映らない原因を切り分ける．V0(コントラスト)→RS→共有データ線→E 線の順に1つずつ潰し、まずは待機表示が確実に出る状態にする
 - － 計測版子機を4台で動かし、親機 GPIO トリガから IR デコード完了までの遅延を `micros()` で収集して MOP1 (30ms 以内) と MOP2 (95% 以上) を数字で出す
 - － 収録したピアノ音を使って MOP4 の音色認識アンケートを実施する
 - － LCD が映るようになれば、歌詞/顔が演奏 tick に追従するか (MOP5) を確認する
 - － 最終発表のデモ演奏の準備
- 達成基準：
 - － LCD1 に顔、LCD2 に歌詞が安定して映り、再現性を持って表示できること
 - － 計測データで子機間のズレが 30ms 以内、IR 受信成功率が 95% 以上であることを数値で提示できること
 - － アンケートで各楽器の音色が過半数に識別されること
- 期限：
 - － 7/8（最終週の締め／最終発表準備）

8. その他

- LCD は手代木が手を離れた状態から引き取ったので、配線の意図（どのピンを何に使っているか）を HAISEN.md にまとめ直すところから始めた．次に触る人が迷わないよう、SC1602B の電源ピンが市販 1602A と逆である点は特に強調して書いた
- 表示子機は音楽子機 (hack-koP2) とは別の Arduino に焼く前提．同じ基板に両方は載らないので、最終発表では表示用と演奏用で基板を分ける
- 計測プログラムは親機 GPIO を子機に直結する配線が前提なので、計測時だけワイヤーを1本足す運用にする．本番の演奏構成では外す
- LCD の実機写真は、安定して映るようになってから最終版のログに載せる（今は映ったり映らなかったりで、載せられる状態の写真が撮れていない）

9. 付録

9.1 添付資料

- ソースコード：テスト版/flogkao/flog_kao_kasi_ir_full/flog_kao_kasi_ir_full.ino（顔＋歌詞の2枚LCD表示子機）、テスト版/hack_koP2saisei_kokitest/hack-koP2.ino（IR伝搬遅延の計測版子機）、配線ガイド HAISEN.md
- リポジトリ：<https://github.com/k-kushihama/hackathon-code/>

9.2 添付範囲

- 第10週に引き取ったLCD表示子機（SC1602B×2、顔＋歌詞）の実装スケッチと配線ガイド

- 第 10 週に整備した IR 伝搬遅延の計測版子機スケッチ（GPIO トリガ基準の `micros()` 差分計測）

9.3 システム図

9.3.1 IR 伝搬遅延の計測方法

図 1 に今週組んだ計測の流れを示す。親機の GPIO をワイヤーで子機の `PIN_SYNC_IN` に直結しておき、親機はまず GPIO を HIGH にする。この立ち上がりの子機の割り込み `onSyncRise()` が拾って `syncRiseUs=micros()` を記録する（計測開始）。GPIO は電気信号なので、このトリガはほぼ遅延 0 で子機に届く。親機はトリガを出した直後に `sendNEC()` で IR（NEC プロトコル）を送出し、子機の IR レシーバ（CHQ1838D）がそれをデコードする。子機は `loop()` でデコード結果を処理した時点で `recvUs=micros()` を取る（計測終了）。差分 `recvUs-syncRiseUs` が「NEC を受信してデコードし `loop` で処理し終えるまで」に要した時間で、これが通信誤差の主成分になる。

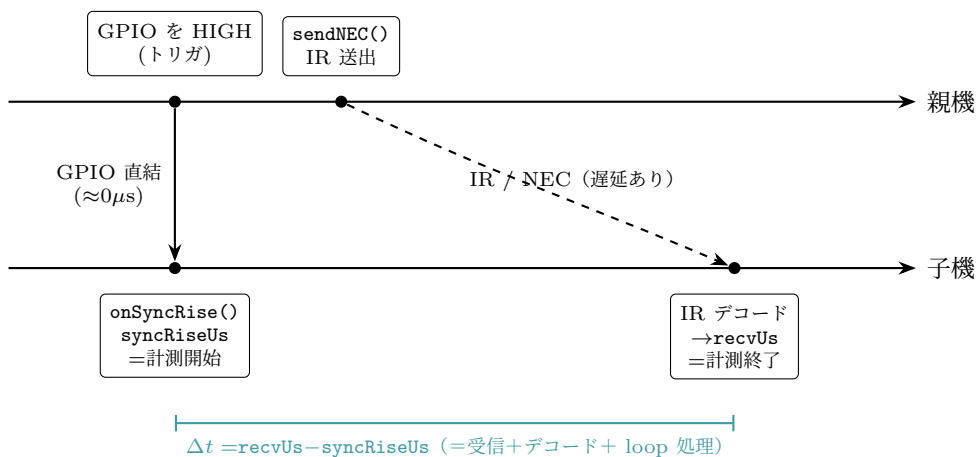


図 1 IR 伝搬遅延の計測方法。GPIO 直結のトリガで計測開始、IR デコード完了で計測終了とし、その差分を通信誤差の主成分として測る。

9.3.2 顔+歌詞 LCD 表示子機の構成

図 2 に、今週引き取った LCD 表示子機の構成を示す。1 台の表示子機 Arduino に SC1602B を 2 枚ぶら下げ、LCD1 に顔、LCD2 に歌詞を出す。RS (D12) とデータ線 DB4-7 (D5,D4,D3,D2) は 2 枚で共有し、E 線だけ LCD1→D11・LCD2→D10 に分けることで、どちらの LCD に書くかを切り替える。IR レシーバ（CHQ1838D）は D6 で受け、親機からの tick に合わせて歌詞送りと口パクを進める。この構成自体は配線図としては成立しているが、実機ではまだ安定して表示できていない。

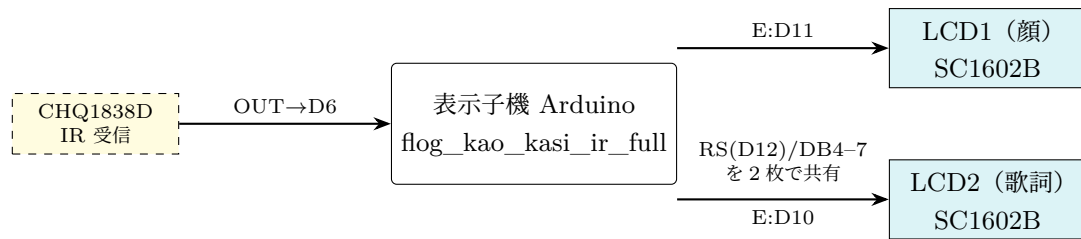


図 2 顔+歌詞 LCD 表示子機の構成. RS とデータ線を 2 枚で共有し, E 線 (D11/D10) で書き込み先を切り替える. IR は CHQ1838D で D6 受け. 配線としては成立しているが実機表示が未安定.